Visitor Pattern - Advanced File System

Sulukhan Aizharykova | SE-2529 | Software Design Patterns | Assignment 06

1. Purpose and object structure

The application models an in-memory file system and applies three independent operations to one tree: total byte size, configurable file search and structural statistics. FileNode stores an immutable name and non-negative long byte size. DirectoryNode owns named child nodes, including nested and empty directories. No real computer files are inspected.

Element is the common interface; sealed abstract Node shares validation and parent ownership. FileNode and DirectoryNode are the only concrete elements. Visitor<R> declares both visit overloads and returns a typed operation result. AbstractVisitor centralizes reset and traversal location. Main builds the dataset and invokes each operation through TreeWalker. The UML is on a separate final page, excluded from the three-page report limit.

2. Double dispatch and traversal

First, element.accept(visitor) dynamically selects FileNode.accept or DirectoryNode.accept. In that concrete method, visitor.visit(this) selects the matching overload using this's compile-time concrete type; virtual dispatch then selects that overload's implementation on the actual visitor. For example, FileNode plus TotalSizeVisitor adds bytes, while DirectoryNode plus that visitor adds nothing. No instanceof, reflection, type-selection casts or branching chains are used.

TreeWalker alone owns traversal. It uses an explicit stack for pre-order depth-first processing, calls accept exactly once per reachable node and pushes each child exactly once. Elements dispatch only; visitors process only the current node. A file exposes no children. Tree integrity prevents shared descendants and cycles, eliminating duplicate visits. The stack supports deep trees without recursive Java calls.

3. Paths, depth and reusable state

The selected root has depth 0 and an internal empty relative path. Child depth is parent depth + 1; paths append the child name with /. A directory root's name and all ancestors are omitted, even when selecting a nested subtree. A standalone file root is returned under its filename. Search sorts matching paths in natural lexicographic String order and returns an immutable snapshot.

Every walk resets accumulated state and provides path/depth before dispatch. Visitor instances can be reused sequentially, including after a failed run. Previous search lists and Statistics records remain unchanged. Visitors are not shared concurrently or reentrantly, and tree mutation during a walk is unsupported. Use walk for complete operations; direct accept alone does not run the lifecycle.

4. Integrity and five Clean Code practices

Invalid mutations fail before insertion: names cannot be blank, dot segments, contain separators/control characters or have surrounding whitespace; sizes cannot be negative. DirectoryNode.add rejects duplicate sibling names, existing ownership and ancestor/self cycles. remove detaches a node for explicit movement. Child collections are immutable snapshots. Nulls fail explicitly; size overflow throws ArithmeticException instead of silently wrapping.

Meaningful naming

FileNode.sizeBytes, SearchVisitor.minimumSizeBytes and TreeWalker.Frame.relativePath make units and responsibilities explicit, reducing ambiguity when reading filters and output.

Single Responsibility

DirectoryNode manages ownership, TreeWalker manages traversal and each concrete visitor implements one operation, so changing a search rule does not change tree integrity or traversal.

Small methods

DirectoryNode.rejectCycle and StatisticsVisitor.recordDepth isolate small, named tasks, keeping add and visit short enough to inspect independently.

Encapsulation

DirectoryNode.children returns List.copyOf and Node keeps its parent private with package-private updates, so callers cannot bypass the public ownership rules through writable collections.

Elimination of type-checking chains

FileNode.accept and DirectoryNode.accept call visitor.visit(this), allowing overloaded methods and virtual dispatch to select behavior instead of instanceof chains or casts.

5. Extensibility and trade-offs

The initial contract includes both element types and visit overloads. Commit bd66764 implements the elements, walker, size and search. The next commit, 89f0ff1, adds only Statistics.java and StatisticsVisitor.java; neither element nor either accept method nor the Visitor contract changes. This makes the third operation's extension boundary visible in history.

Adding another operation requires another visitor with both overloads. Adding another element type requires changing the sealed hierarchy, adding a visit overload and updating every visitor and tests. Visitor therefore favors stable element types with evolving operations, at the cost of additional classes and coupling visitors to concrete element types.

Traversal performs O(n) visits; search adds O(m log m) sorting for m matches. Complete path construction and pending frames also cost space/time proportional to the relevant path lengths. Immutable snapshots simplify safe result reuse but allocate copies. This small project uses single-threaded ownership rather than concurrency controls.

6. Executable sample and changed configuration

The custom workspace contains 7 files and 5 directories, including assets/empty. Its file sizes are 8192, 1024, 256, 128, 2048, 0 and 512 bytes. The deepest files are under docs/drafts at depth 3. Default execution prints:

```
Root: workspace
Total size: 12160 bytes
Search: suffix=.txt, minimum=256 bytes
  docs/drafts/notes.txt
  docs/guide.txt
Files: 7
Directories: 5
Maximum depth: 3
Repeated search unchanged: true
```

Run mvn test for all tests. Run mvn package, then java -jar target/visitor-filesystem-1.0.0.jar for the application. Adding arguments .md 128 produces README.md and docs/drafts/todo.md, with the same size and statistics. Search combines the case-sensitive filename predicate AND size >= threshold; notes.txt demonstrates the inclusive boundary.

7. Automated verification

JUnit 5 via Maven Surefire: 41 tests run, 0 failures, 0 errors, 0 skipped. TreeIntegrityTest: 18 invocations, including 9 invalid-name parameters. VisitorOperationsTest: 18 tests. ApplicationTest: 3 tests. DispatchTest: 2 tests. Test fixtures use independently chosen data, not only the demonstration tree.

Coverage includes matching visit overloads (T1), nested size totals (T2), both search constraints (T3), sorted root-relative paths (T4), counts and depth including empty directories (T5), empty/no-match cases (T6), repeat runs and immutable snapshots (T7), and rejected mutations with valid nesting/movement (T8). Additional tests cover a file root, zero bytes, 2000-level nesting, overflow and recovery after a visitor failure.

Local verification used Maven 3.9.9 and JDK 25 targeting release 17. GitHub Actions also passed the suite and both application configurations on Temurin 17. Compiler warnings are errors. The executable output is asserted in ApplicationTest.

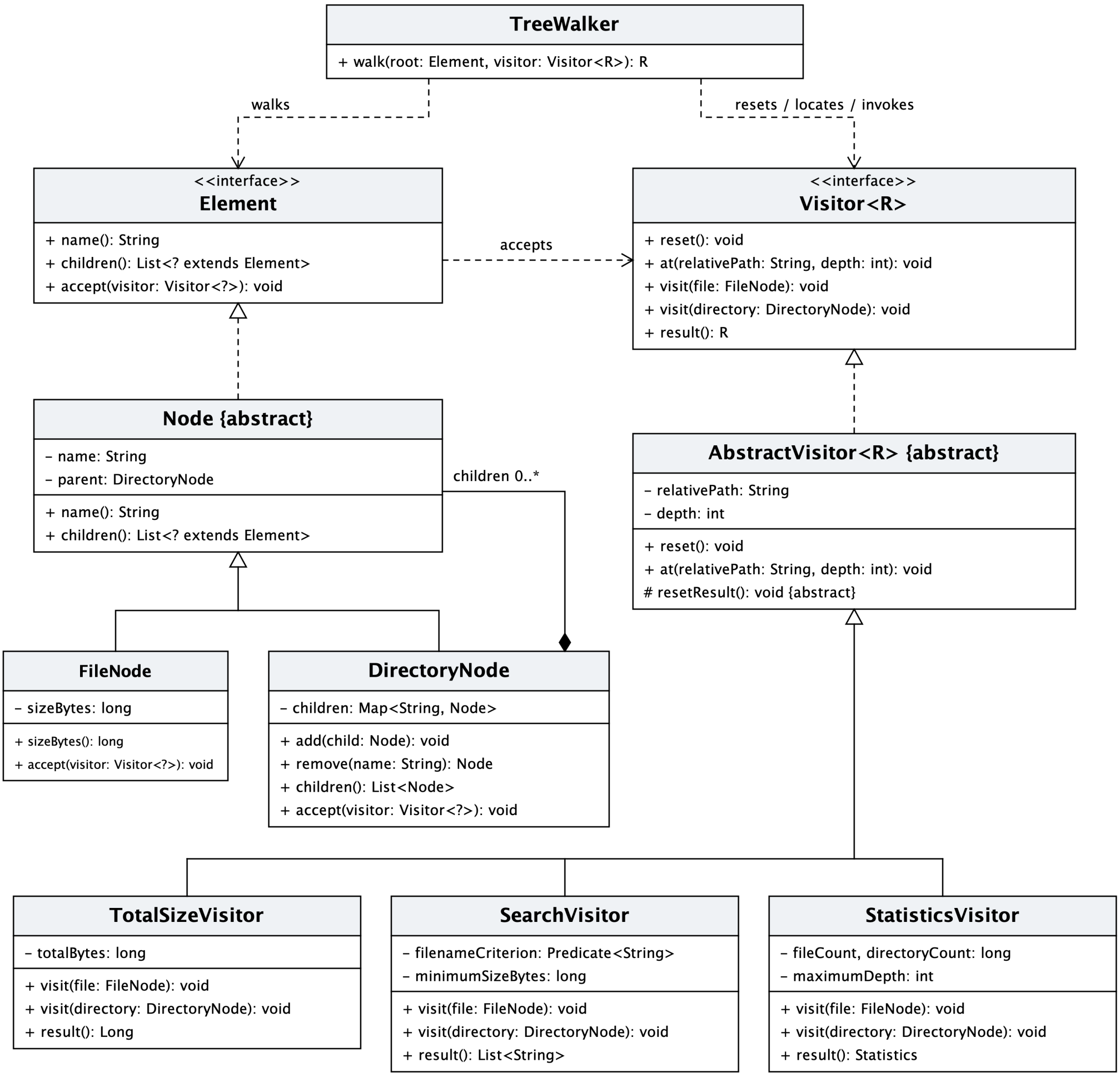
8. Repository, sources and assistance

https://github.com/sulumate/spd6_sulumate

Sources: Assignment 06 handout; Java Language Specification 17, section 15.12 (method invocation); Apache Maven, Introduction to the Build Lifecycle; JUnit 5.11.4 User Guide. Full source URLs are in README.md.

ChatGPT/Codex assisted with code, tests, report and UML. The student must review and understand the implementation and follow the course's independent-work policy. Repository completion does not replace the individual defense.

UML appendix - Visitor Pattern



Visitor extension: StatisticsVisitor was added without changing elements or accept methods.

Dashed triangle: implements. Solid triangle: extends. Filled diamond: owned children.

Only pattern-relevant members are shown; constructors and private helpers are omitted.

The three report pages precede this separate UML appendix.